Date: __/__/__ Day

CHAPTER : 3

★ "VECTORS (DYNAMIC ARRAYS)" ★

VECTORS :-

Definition :-

"Vectors are dynamic arrays which can double their size on runtime if they run out of space".

Explanation :-

The normal array is allocated memory at compile time and this memory remains fixed during runtime. Consider an array:

```
int arr[10];
```

This array can only store 10 elements and no matter what we do, we cannot insert an 11th element into this array.

This is where vectors come in. Data is inserted into a vector using the vector's push_back(int n) method. To study a vectors working, consider a

Date: / / Day

vector :-

```
vector arr(5);
```

This statement creates a vector in the memory of size 5. Now let's insert elements using push_back :-

```
arr.push_back(1);
arr.push_back(2);
arr.push_back(3);
arr.push_back(4);
arr.push_back(5);
```

By now, our vector is completely filled. What would happen if we did push_back one more time? Here is what happens :-

1) Another vector of double size of arr forms in memory.
2) Elements of arr are copied into that new vector.
3) Old vector arr is deleted.
4) This new vector of size 10 becomes arr.
5) 6th element is then inserted into arr with 4 empty places remaining.

Date: __/__/__ Day

To study this more thoroughly, we need to take a look at the internal structure of a vector, which is built using a class.

Code :-

```
class vector
{
private:
    int *ptr;
    int cap;
    int length;
public:
    vector(int cap=0):cap(cap), length(0){
        ptr = new int[cap];
    }
    ~vector(){ delete[] ptr; }
    // Some other methods like:
    // push_back, resize, pop_back etc ...
};
```

The memory diagram of a vector is the same as array. Vector also occupies consecutive memory boxes under a single identifier name.

Now let's look into the internal structure and working of a vector.

Date:__/__/__ Day_____

DATA MEMBERS :-

Lets take a look at the data members :-

int * ptr;

This is the array pointer used for making a dynamic array. When a vector is declared, memory is allocated to ptr in the constructor. The destructor frees this memory.

int cap;

This cap is short for capacity and it keeps track of the current size of ptr array. Capacity is also the maximum number of elements that the vector can currently hold. Value of cap double when a vector resizes and doubles its size. Cap is passed to constructor during declaration of vector, otherwise it is set to 0 by default.

int length;

This length keeps track of current number of elements in array. Its value starts off as zero and increases / decreases with every insertion/deletion.

Date: __/__/__ Day _____

METHODS :-

Now lets take a look at the methods (member functions) of a vector :-

Constructor :-

Constructor is automatically called when a vector is declared and the value of capacity is passed to it during declaration, if not then it is set to 0 by default. The constructor does two more tasks :-

1) Sets length to 0.
2) Allocates memory of cap size to ptr.

Destructor :-

The destructor is automatically called when the scope of a vector ends. The job of destructor is simple. It deallocates the memory which was allocated to the ptr by the constructor when vector was declared. Thats it.

Now lets take a look at major functions of a vector which are associated with its working and implementation.

Date: __/__/__ Day _____

PUSH_BACK :-

Push_back method is to insert an element at the back of the vector, it takes a single variable as argument which should be of the same datatype as *ptr pointer. Following is the code of vectors push_back method.

Code :-

```
void push_back(int value){
    if (length == cap) resize();
    ptr[length++] = value;
}
```

It checks if vector is full by comparing length (current elements) to cap (max elements). If they are equal then vector is full and should be resized before inserting the element. If it is not full then there is no need to resize since vector still has space. Either way, final step is to insert the new element at length index and then update length using length++ to update the current number of elements.

Date: __/__/__ Day_____

RESIZE :-

Resize method is used to double the size of ptr array at runtime by doubling the value of cap. Resize is only called if vector is full and a new element is to be inserted. Following is the code of resize method :-

Code :-

```
void resize(){
    if(cap==0) cap++;
    else       cap*=2;
    int * newPtr = new int[cap];
    for(int i=0; i<length; i++)
        newPtr[i]=ptr[i];
    delete[] ptr;
    ptr = newPtr;
}
```

If you compare this code with the process of what happens when a push-back is called on a filled vector, you will see the first four steps of that process in action. So go back 5 pages and compare this code with those 4 steps.

Date:__/__/__ Day_____

POP_BACK :-

Pop-back method is used to remove the element at the back of the vector. It is basically opposite of a push-back. It checks if length is not 0, meaning the vector is not empty so there is an element to remove. Then it just decreases the value of length by 1 to make the last element go out of access, making it look like as if it was removed. Following is the code of pop-back method.

Code :-

```
void pop_back(){
    if(length!=0)
    {
        length--;
    }
}
```

This length-- updates length so that current number of elements decreases by 1 and last index becomes in accessible as well.

Date: __/__/__ Day _____

CAPACITY :-

Capacity method is used to get the current capacity which is "cap".

Code :-

```
int capacity() const { return cap; }
```

SIZE :-

Size method is used to get the current size which is "length".

Code :-

```
int size() const { return length; }
```

EMPTY :-

Empty method is used to check is vector is empty or in other words, if length is 0.

Code :-

```
bool empty() const { return length == 0; }
```

Date: __/__/__ Day _____

SHRINK_TO_FIT :-

Shrink-to-fit method is used to make the capacity (cap) equal to size (length). It frees up any unused space in the vector. Following is the code of shrink-to-fit method.

Code :-

```
void shrink_to_fit(){
    cap = length;
    int* newPtr = new int[cap];
    for(int i=0; i<length; i++)
        newPtr[i] = ptr[i];
    delete[] ptr;
    ptr = newPtr;
}
```

Its working is pretty similar to resize method but this one reduce cap to length instead of doubling it.

This is all for vectors, there are other methods but these are major ones. You can find the codes on github : "github.com/coder-Retro/Fast-University" Go into DSA > Vector, GOOD LUCK!